

Large-Scale and Multi-Structured Databases

ACID vs BASE

Prof Pietro Ducange

The Tasks of DBMSs

Databases must allow users to store and retrieve data. To this aim, three tasks must be in charge to DBMSs:

- Store data ***persistently***
- Maintain data ***consistency***
- Ensure data ***availability***

Distributed systems

Most of the recent NoSQL DBMSs can be deployed and used on ***distributed systems***, namely on ***multiple servers*** rather than a single machine.

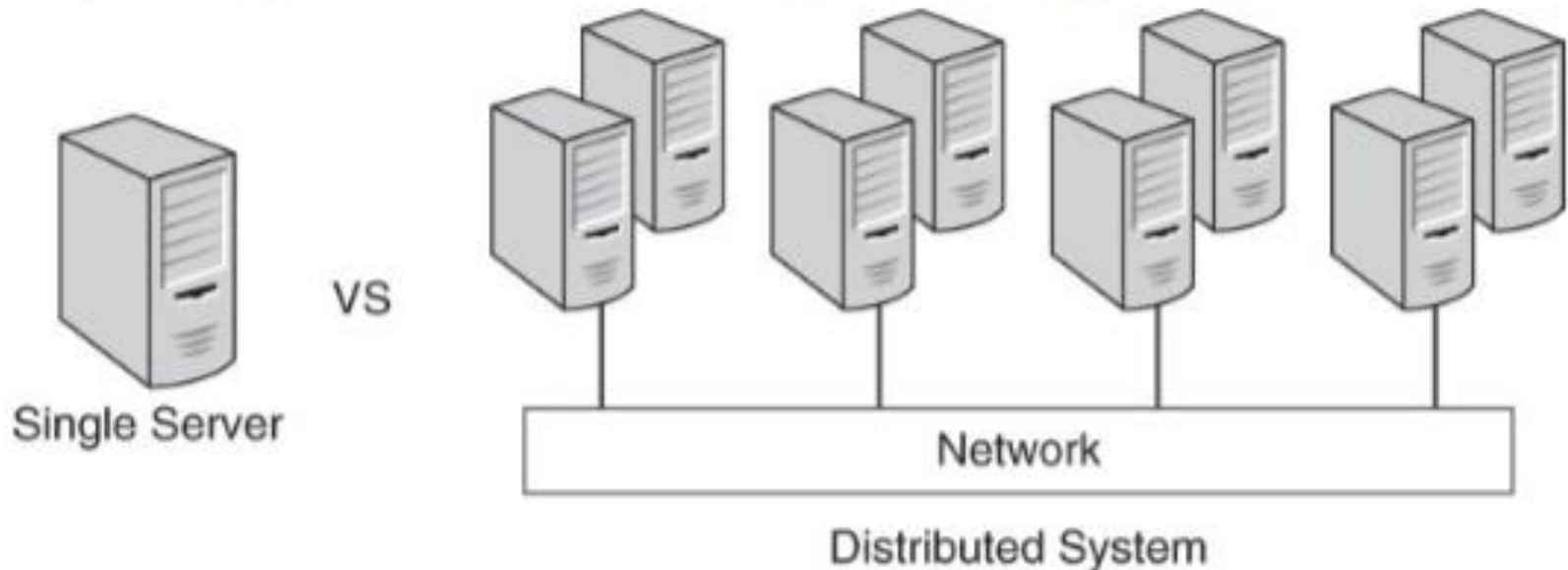


Image extracted from "Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015"

Pros of Distributed Systems

- Ensure ***scalability, flexibility, cost control*** and ***availability***
- It is easier to add or to remove nodes (***horizontal scalability***) rather than to add memory or to upgrade the CPUs of a single server (***vertical scalability***)
- Allow the implementation of ***fault tolerance*** strategies
- Accomplish with the ***motivations*** that led to the third databases revolution!

Some Cons of Distributed Systems

- To **balance** the requirements of data **consistency** and system **availability**
- To protect themselves from **network failures** that may leave some nodes **isolated**

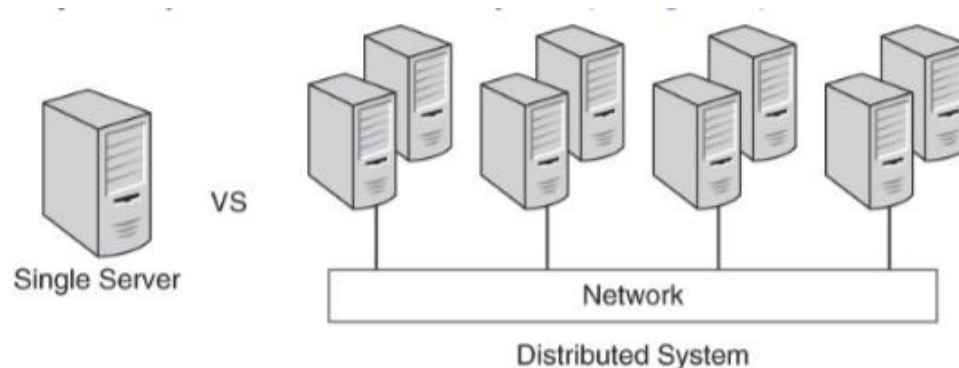


Image extracted from "Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015"

Data Persistency

Data must be stored in a way that is ***not lost*** when the database server is shut down.

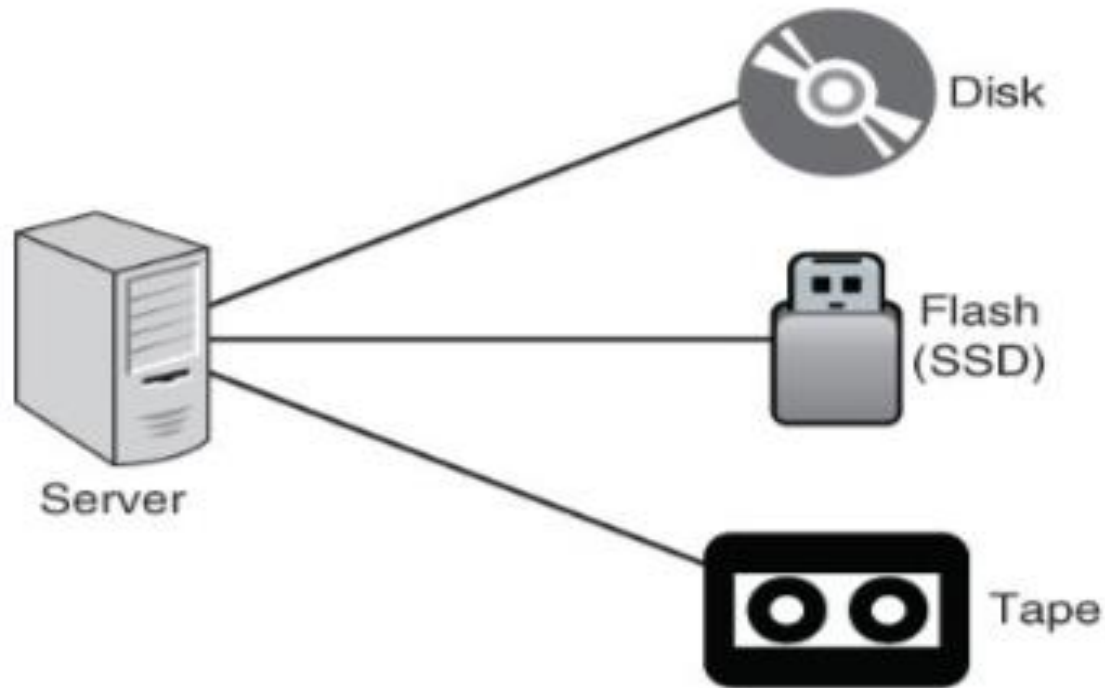
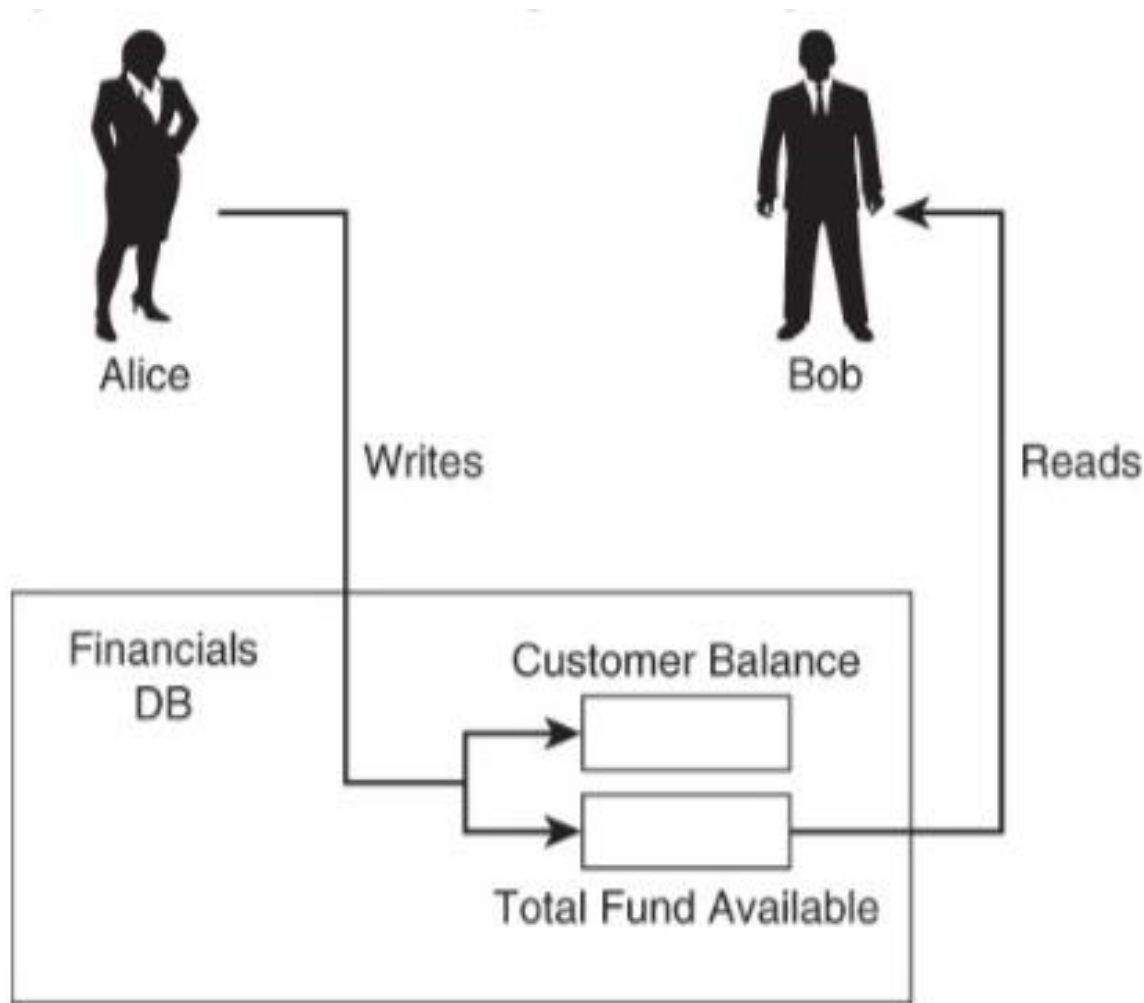


Image extracted from "Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015"

Data Consistency and Integrity



Consistency and **integrity** of a database describe a state in which the stored data does not contradict itself.

Consistency ensures that a transaction can only bring the database from one **valid** state to another.

Integrity constraints are to ensure that data **consistency** is **maintained** for all insert and update operations.

Image extracted from "Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015"

Data Consistency and Integrity

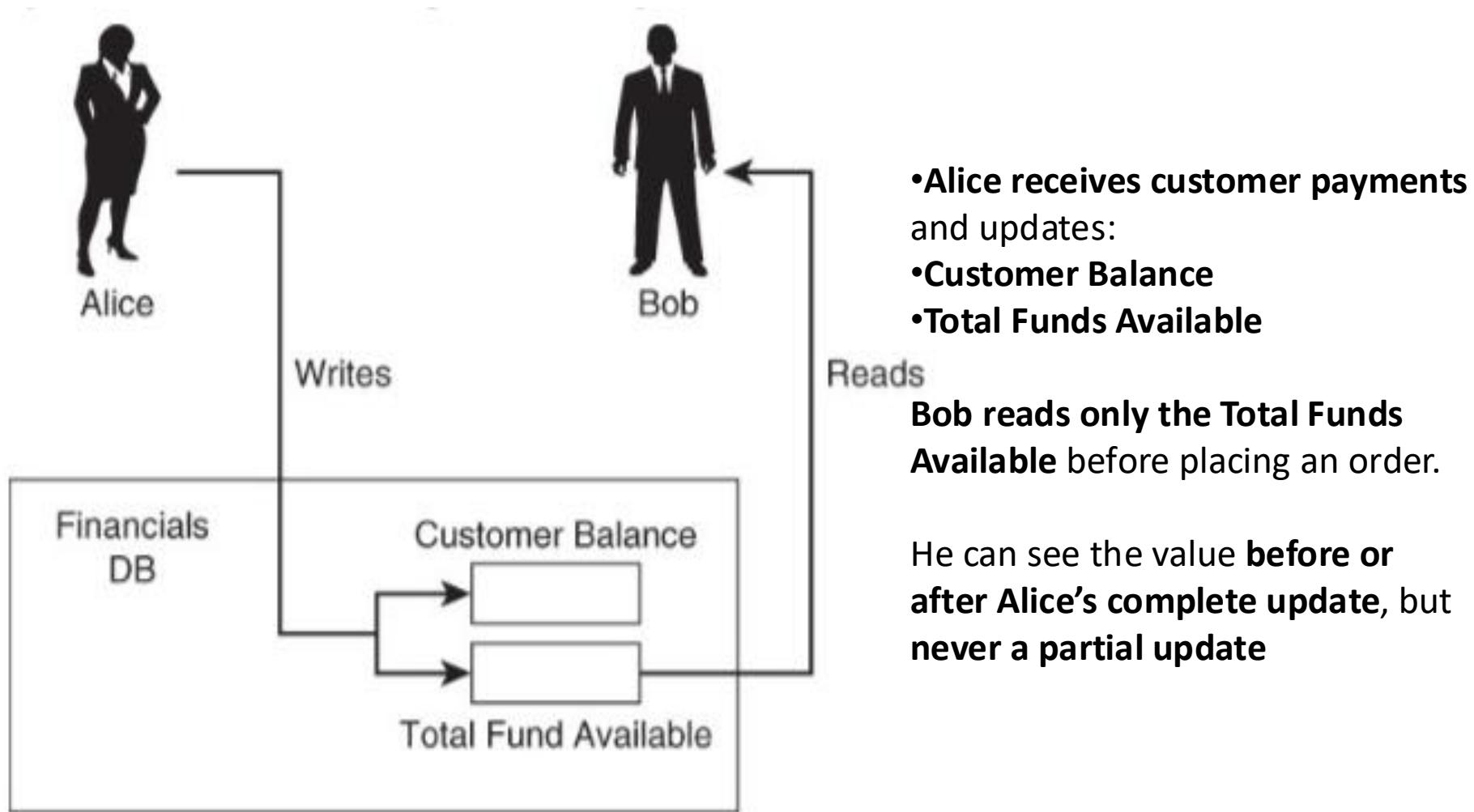


Image extracted from "Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015"

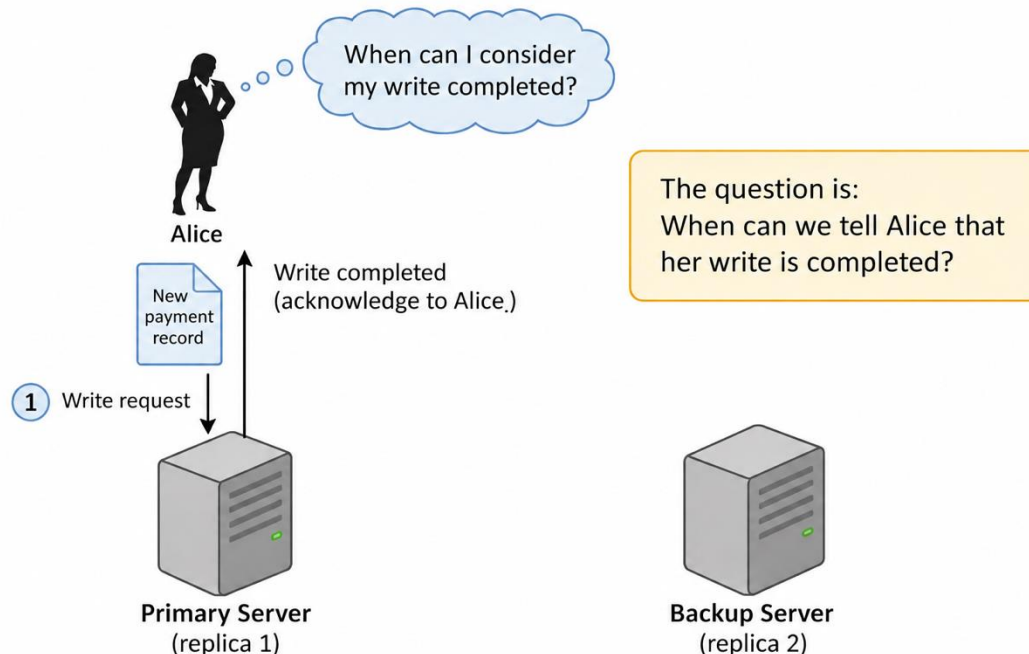
Data Replication

Data stored in a single server DBMS may be not available for several reasons, such as failures of the operating system, voltage drops, disks break down.

A common solution for improving data availability is **replication**: data is maintained on multiple servers.

If the primary server **goes down**, the **Backup server** takes it places and the DBMS continues to offer its **services** to the users.

When is a write completed?



Two-phase commit

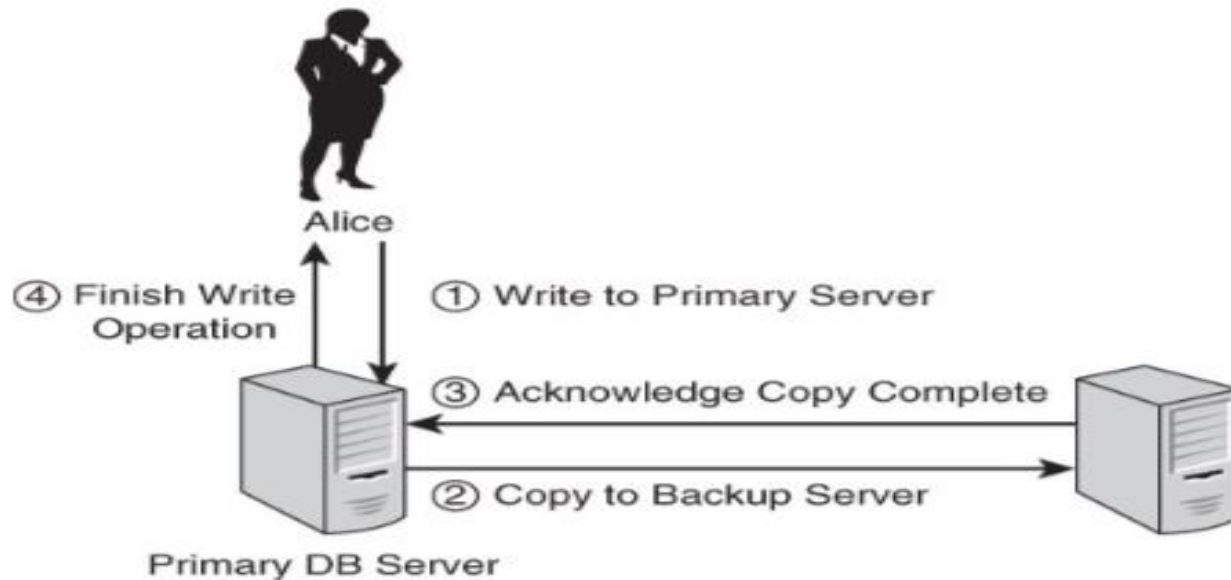


Image extracted from “Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015”

If the primary server **goes down**, the **Backup server** takes its place and the DBMS continues to offer its **services** to the users.

How long it takes the two-phase commit?

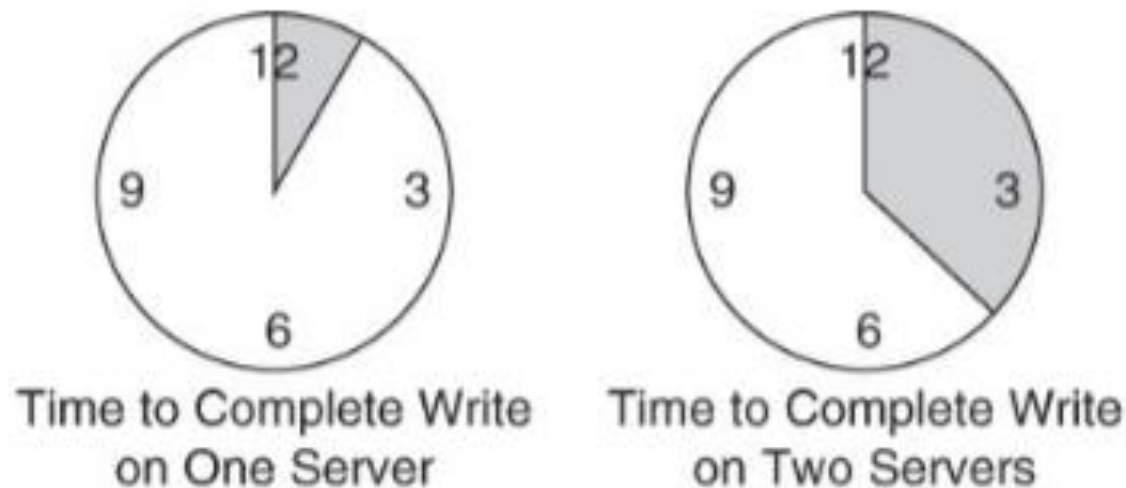


Image extracted from “Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015”

Pay attention: the two-phase commit is a transaction!

You can have consistent data, but ***transactions will require longer times*** to execute than if you did not have those requirements.

ACID Transactions (I)

Jim Gray definition: “A *transaction* is a transformation of state which has the properties of *atomicity* (all or nothing), *durability* (effects survive failures) and *consistency* (a correct transformation).”

An ACID transaction should be:

Atomic: The transaction is ***indivisible***—either all the statements in the transaction are applied to the database or none are.

Consistent: The database remains in a consistent state ***before*** and ***after*** transaction execution.

ACID Transactions (II)

Isolated: While multiple transactions can be executed by one or more users simultaneously, one transaction should ***not see the effects*** of other in-progress transactions.

Durable: Once a transaction is saved to the database, its changes are expected to ***persist*** even if there is a ***failure*** of operating system or hardware.

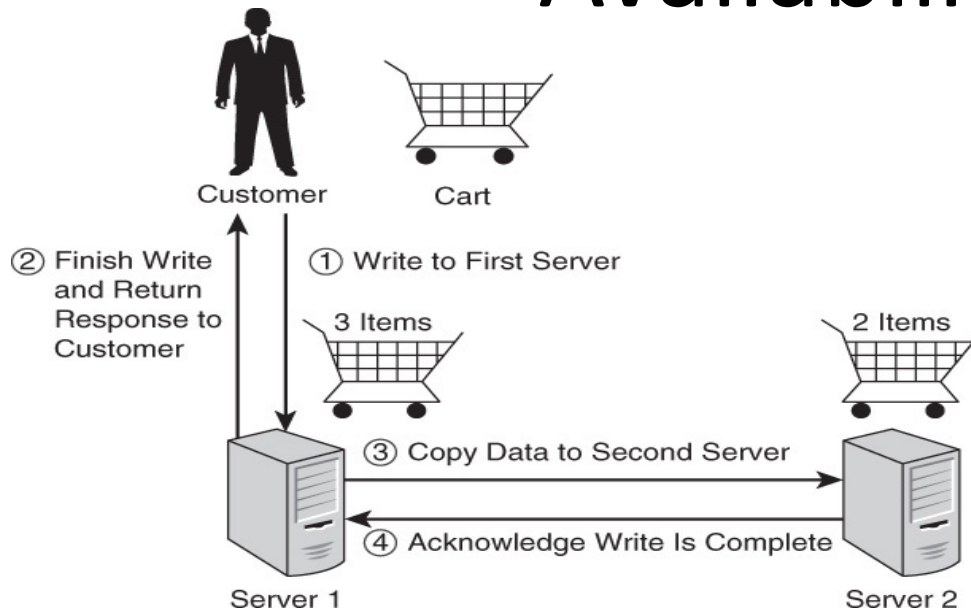
TOO ACID!

There are some applications in which is not acceptable ***waiting too much*** time highly available systems.

In this kind of applications, the availability of the system and its ***fast response*** is more important than having consistent data on the different servers.

Practical example: ***e-commerce website***

Availability First!



NoSQL databases often implement ***eventual consistency***!

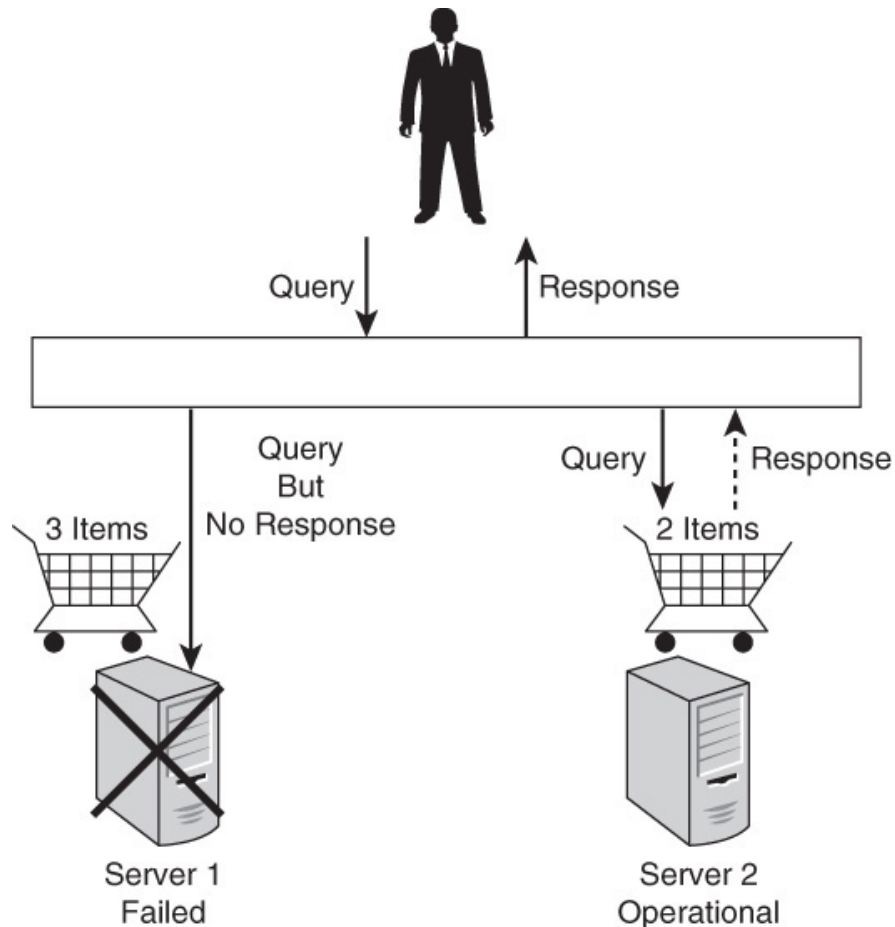
There might be a period of time where copies of data have different values, ***but eventually all copies will have the same value.***

Image extracted from "Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015"

Amazon: even an "insignificant" tenth of a second in the response time of the site generates a reduction in sales estimated at around 1%;

Google: a measly half-second increase in latency caused by traffic can generate a 20% loss of requests.

Available but not Consistent



In the e-commerce scenario the shopping cart may have **a backup copy** of the cart data that is out of sync with the primary copy.

The data would still be **available** if the primary server failed.

The data on the backup server would be **inconsistent** with data on the primary server if the primary server **failed prior** to updating the backup server.

Image extracted from "Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015"

Consistent but not Available (High Latency)

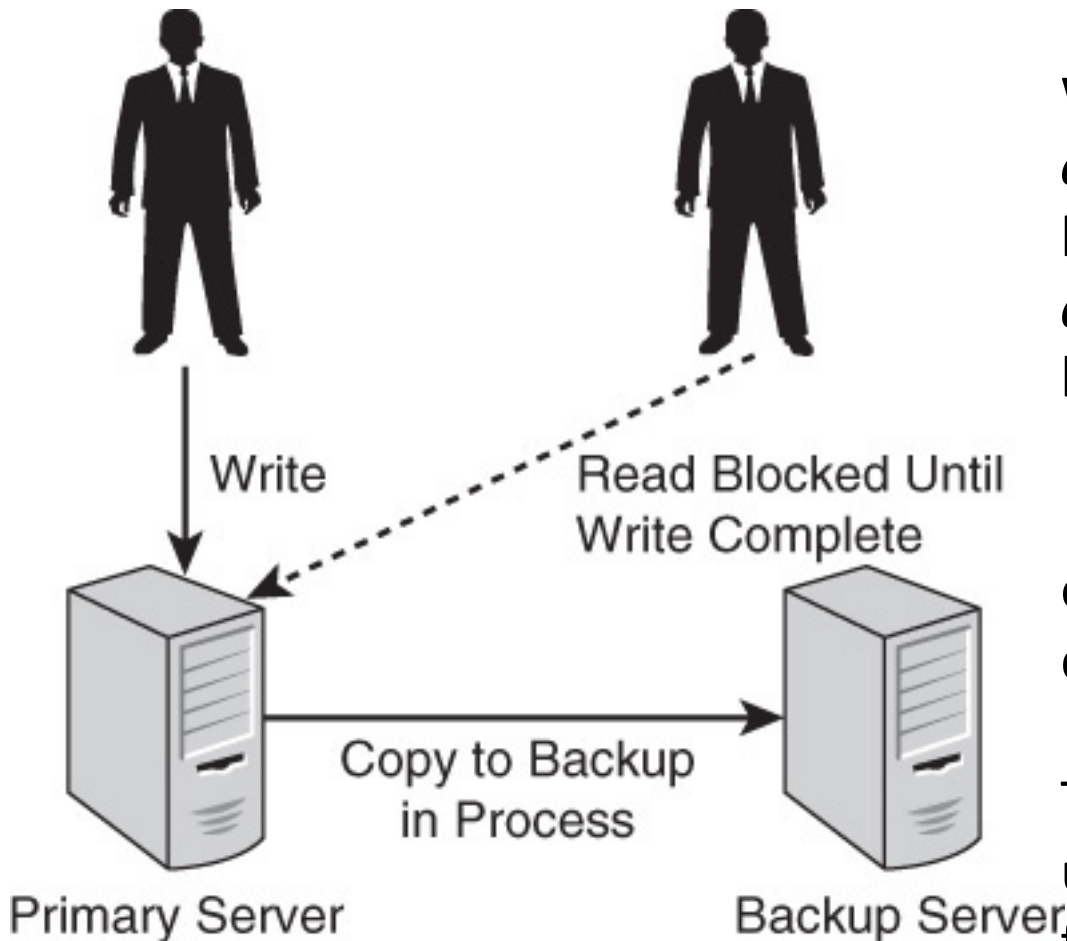


Image extracted from "Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015"

When dealing with **two-phase commits** we can have consistency but at the risk of the most recent **data not being available** for a brief period of time.

While the two-phase commit is executing, other **queries** to the data are **blocked**.

The updated data is unavailable until the two-phase commit finishes. **This favors consistency** over availability

The CAP Theorem (Brewer's theorem)

Distributed Databases cannot ensure at the same time:

- **Consistency (C)**, the presence of consistent copies of data on different servers
- **Availability (A)**, namely to providing a response to any query
- **Partition protection (P)**, Failures of individual nodes or connections between nodes do not impact the system as a whole.

At maximum **two** of the previous features may be found in a distributed database.

The CAP Triangle

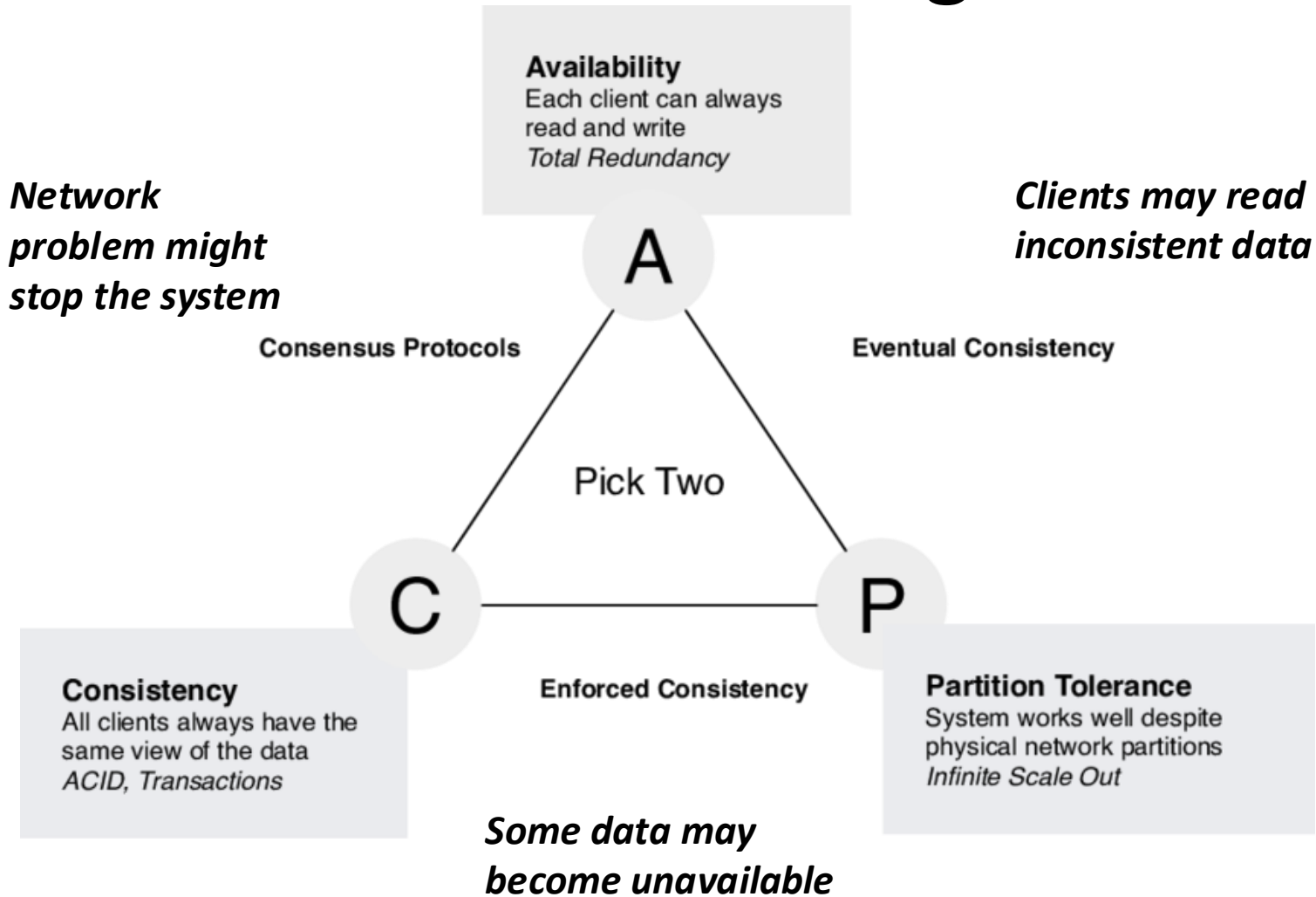
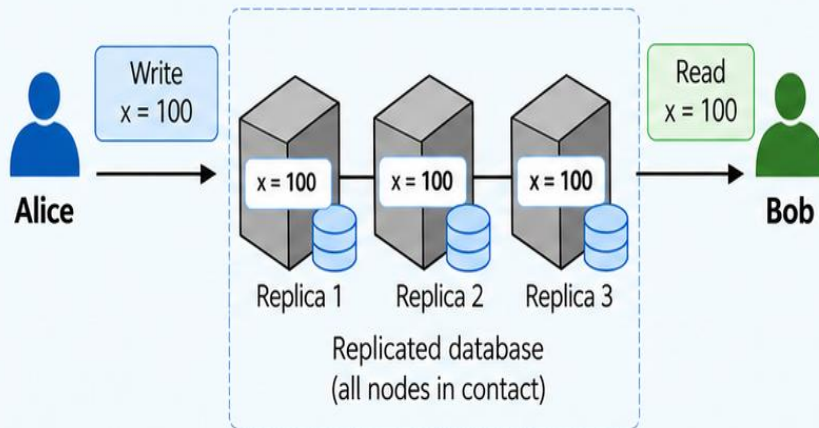


Image extracted from: <http://toppertips.com/cap-theorem/>

CA Solutions

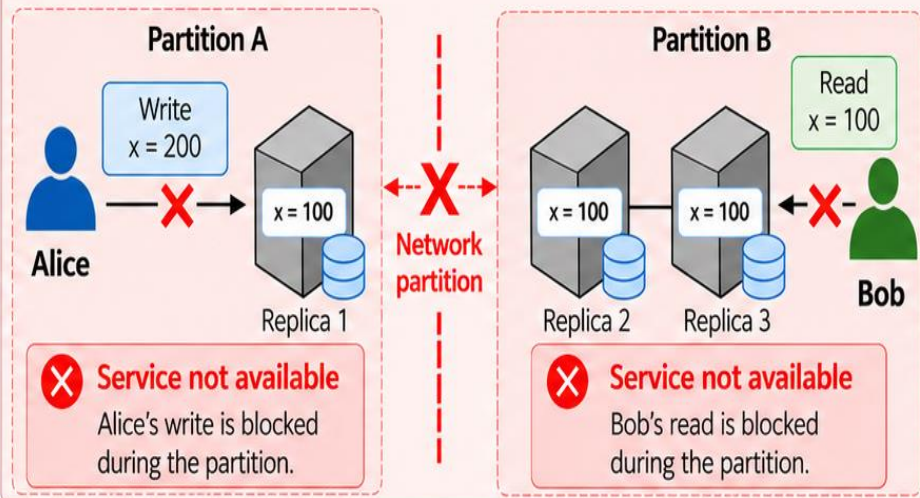
Normal operation (no network partition)

All nodes are in contact and data is consistently replicated.



Network partition (nodes not in contact)

The system cannot guarantee consistency, so it blocks all operations to preserve consistency.



Key properties (CA)

- **Consistency (C):** all clients see the same, up-to-date data.
- **Availability (A):** every request receives a response (as long as no partition occurs).
- **Partition tolerance (P) is not guaranteed:** when a network partition occurs, the system cannot serve requests (it blocks).

Behavior during partitioning

- All nodes cannot communicate.
- To preserve consistency, both read and write operations are blocked for all clients.
- The system remains unavailable until the partition is resolved.

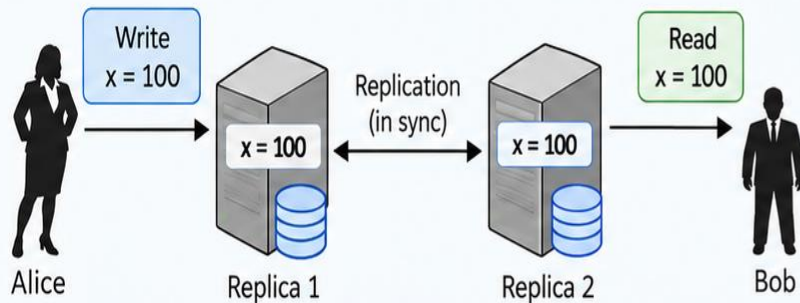
Typical use cases

- Applications where strong consistency is essential (e.g., financial and banking systems, transactional applications).
- Typically deployed in single-site or tightly coupled environments where network partitions are rare and considered exceptional.
- Strong consistency may introduce higher latency.

AP Solutions

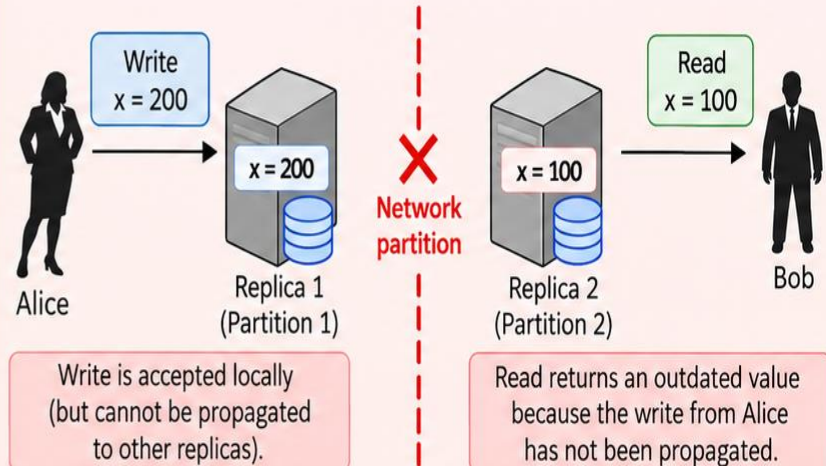
Normal operation (no network partition)

All nodes are in contact and data is replicated.



Network partition (nodes not in contact)

The system is still available, but replicas are not synchronized.



Key properties (AP)

- **Availability (A):** every request receives a response, even during a partition.
- **Partition tolerance (P):** the system continues to operate despite network partitions.
- **Consistency (C) is compromised:** some data returned may be inaccurate (stale).

Behavior during partitioning

- Writes are accepted and stored locally.
- Replicas cannot exchange updates.
- Reads may return stale data (not the most recent version).
- Updates are propagated and replicas become consistent again when the partition is resolved (eventual consistency).

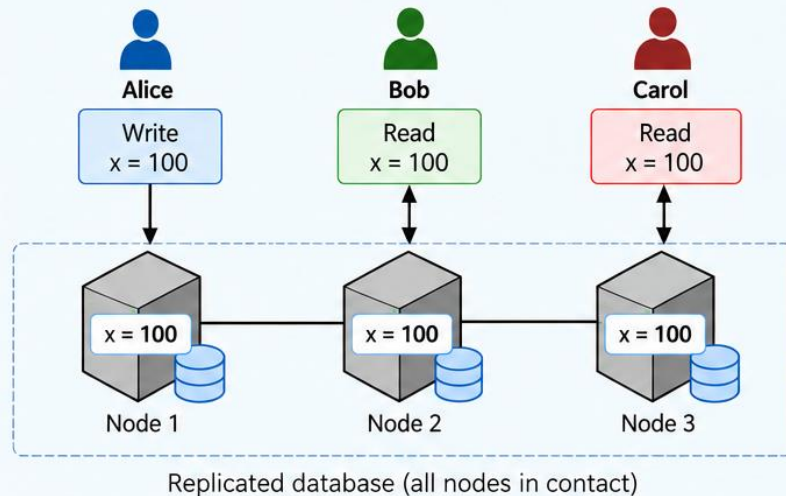
Typical use cases

- Shopping carts (e-commerce)
- News publishing CMS
- Systems that need to continue to function in spite of external errors.

CP Solutions

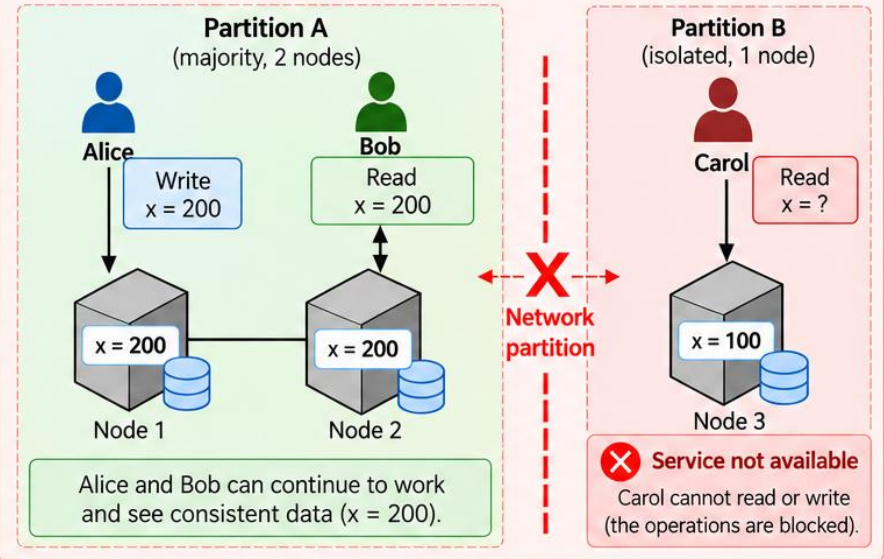
Normal operation (no network partition)

All nodes are in contact and data is replicated consistently.



Network partition (one node isolated)

The system preserves consistency, but some operations are not available.



Key properties (CP)

- **Consistency (C):** all clients see the same, up-to-date data (no stale reads).
- **Partition tolerance (P):** the system tolerates network partitions.
- **Availability (A) is compromised:** during a partition, some operations may not be available (they block).

Behavior during partitioning

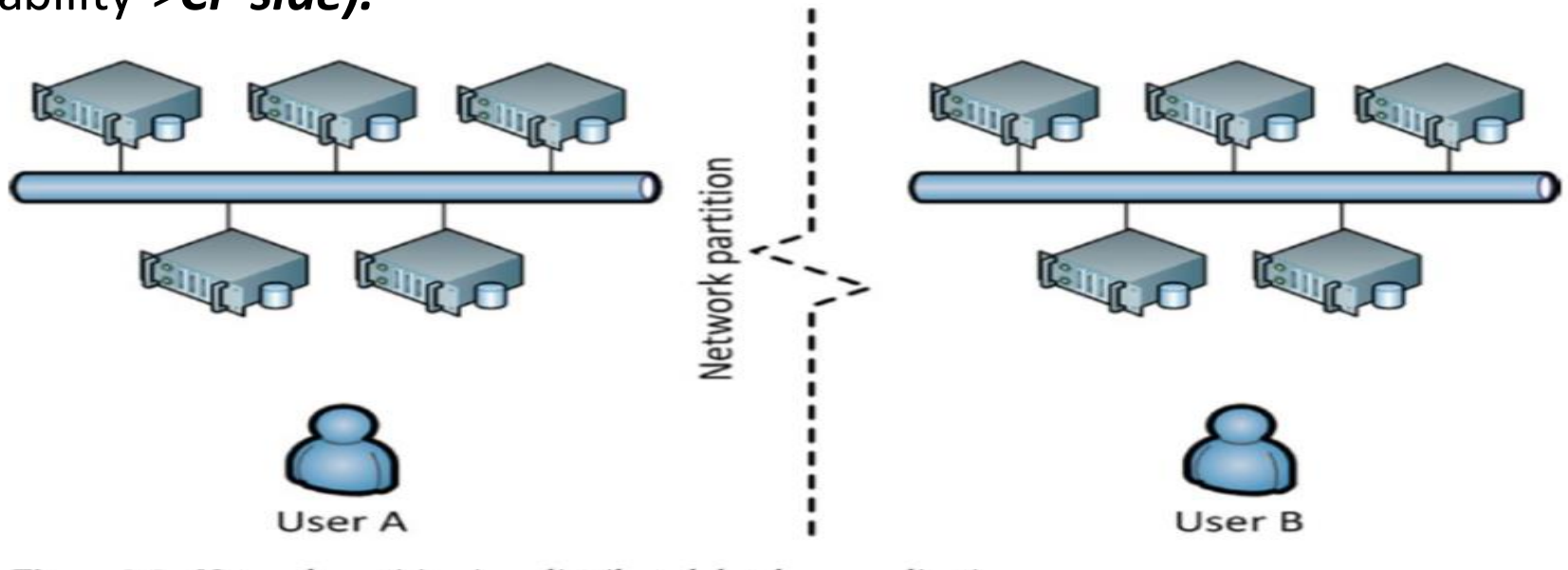
- A majority of nodes remains available: clients connected to it can read and write consistent data.
- Nodes in the minority partition cannot process reads or writes (operations are blocked) to avoid returning inconsistent data.
- When the partition is resolved, the system re-synchronizes and continues with consistent data.

Typical use cases

- Applications that require consistency and partition tolerance, while longer response times are acceptable (e.g., Bank ATMs).
- Usually based on distributed and replicated relational or NoSQL systems supporting CP.

Network Partition Resilience

The system administrator has **two choices** either show each user a different view of the data (availability but not consistency->**AP side**) or shut down one of the partitions and disconnect one of the users (consistency but not availability->**CP side**).



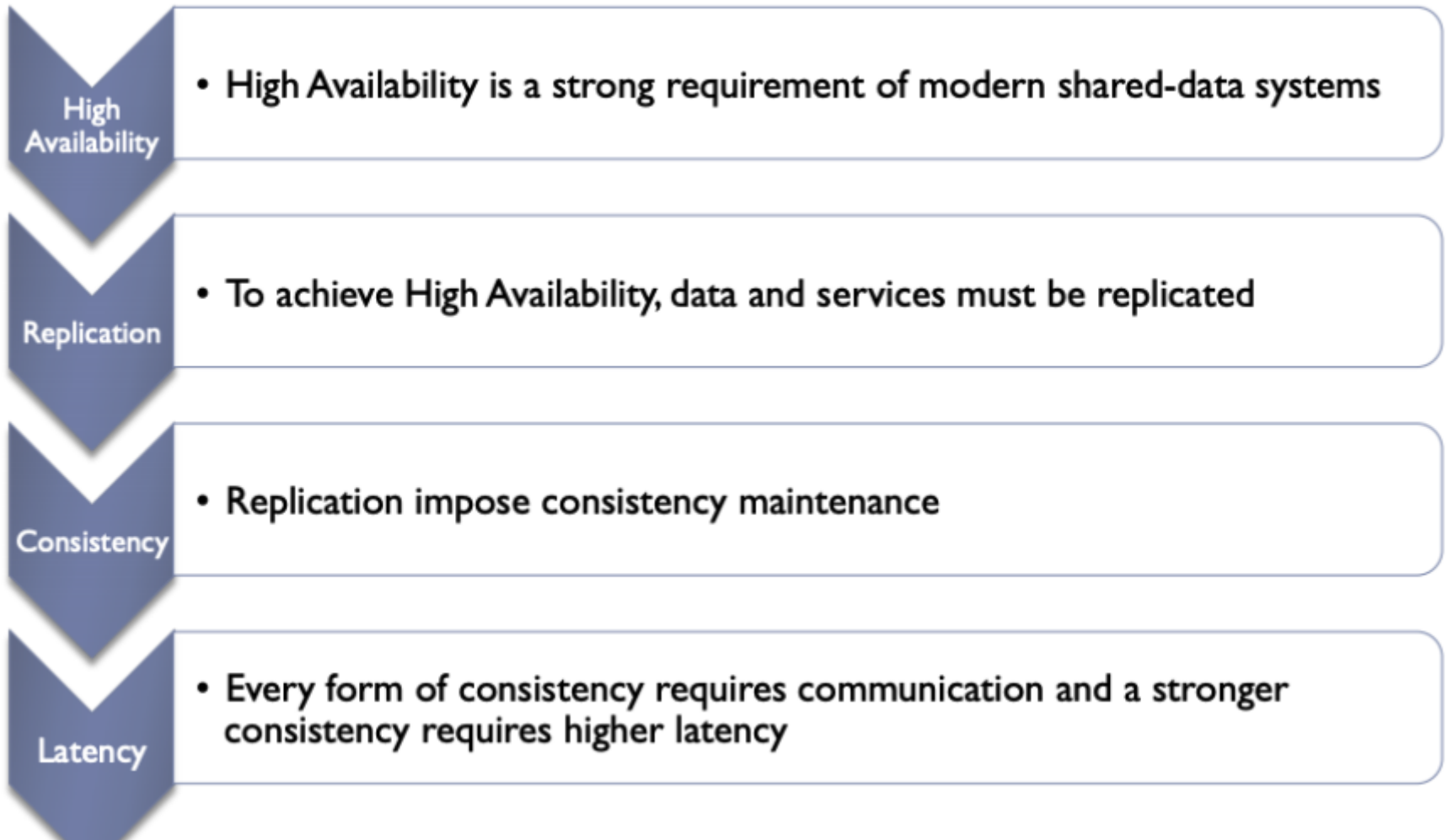
Very important: **AC side is not allowed in this case!**

Image extracted from "Guy Harrison, Next Generation Databases, Apress, 2015"

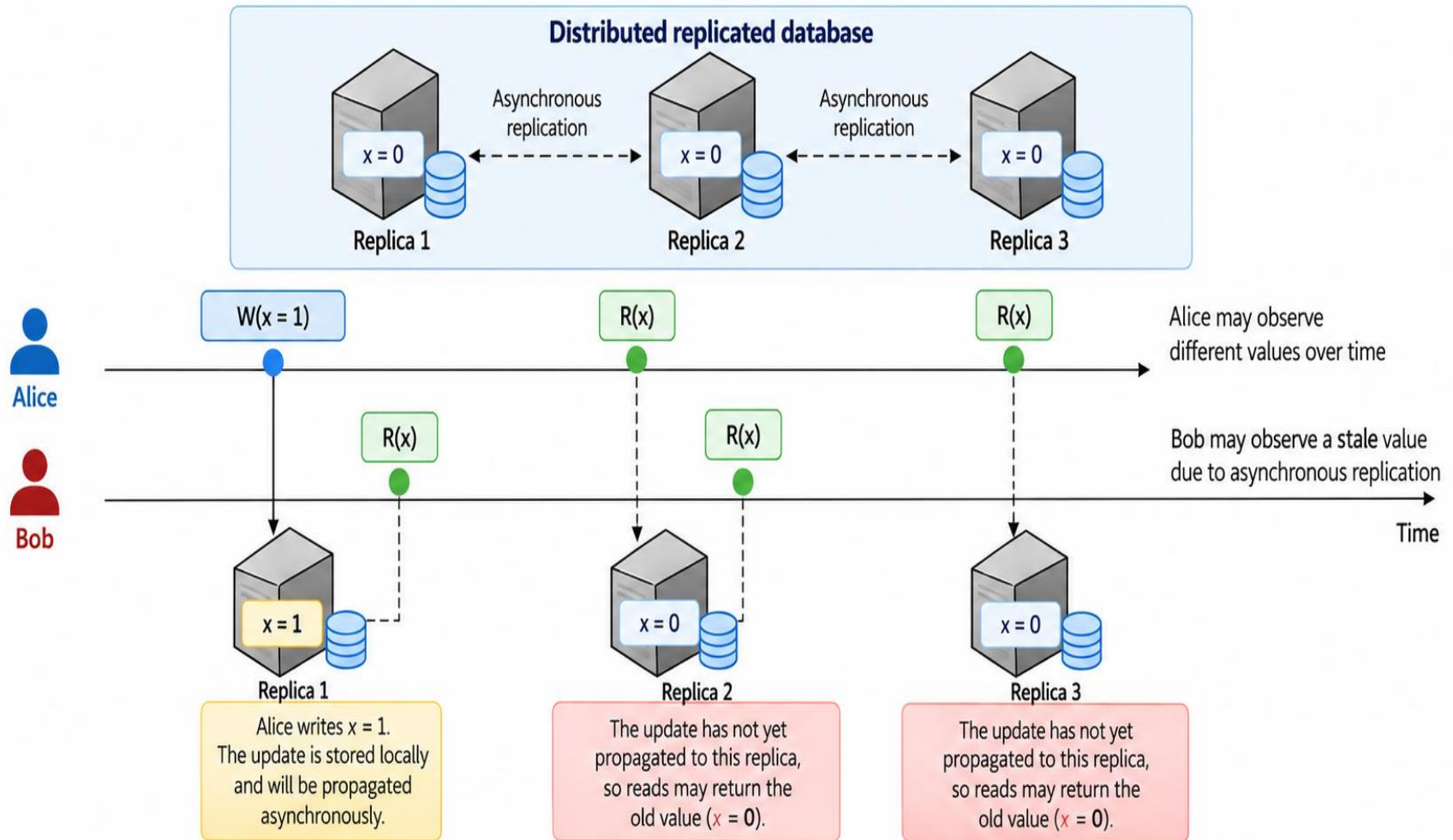
BASE Properties of NoSQL Databases

- **BA** stands for ***basically available: partial failures*** of the distributed database may be handled in order to ensure the ***availability*** of the service (often thanks to data ***replication***).
- **S** stands for ***soft state***: data stored in the nodes may be ***updated*** with ***more recent*** data because of the eventual consistency model (no user writes may be responsible of the updating!!).
- **E** stands for ***eventually consistent: at some point in the future***, data in all nodes will converge to a consistent state.

The Latency Issue



Eventual Consistency: General Setting



Due to asynchronous replication, different users may read different versions of the data.
The system must provide specific guarantees depending on the type of eventual consistency.

Types of Eventual Consistency

- **Read-Your-Writes Consistency** ensures that once a user has updated a record, all of his/her reads of that record will return the updated value.
- **Session consistency** ensures “read-your-writes consistency” during a session. If the user ends a session and starts another session with the same DBMS, there is no guarantee the server will “remember” the writes made by the user in the previous session.
- **Monotonic read consistency** ensures that if a user issues a query and sees a result, all the users will never see an earlier version of the value.
- **Monotonic write consistency** ensures that if a user makes several update commands, they will be executed in the order he/she issued them.
- If an operation **causally depends** on a preceding operation, there is a causal relationship between the operations. **Causal consistency** ensures that users will observe results that are consistent with the causal relationships.

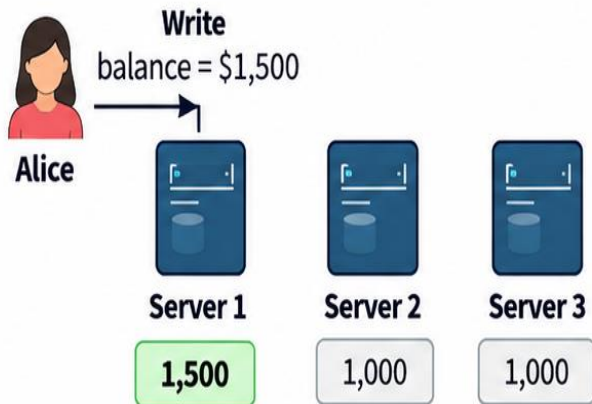
Read-Your-Writes Consistency

A client is guaranteed to see the value of its own writes, even if the write has not yet been replicated to all servers.

Example: Alice updates a customer's balance to \$1,500 and then reads it shortly after.

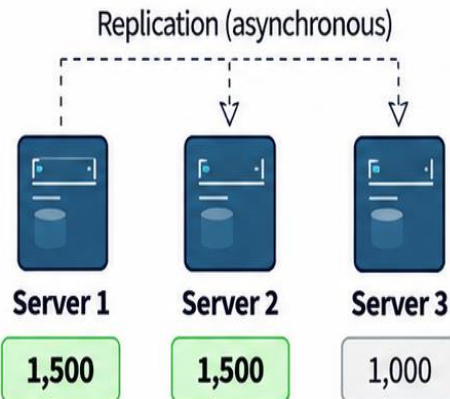
1

Alice writes to Server 1



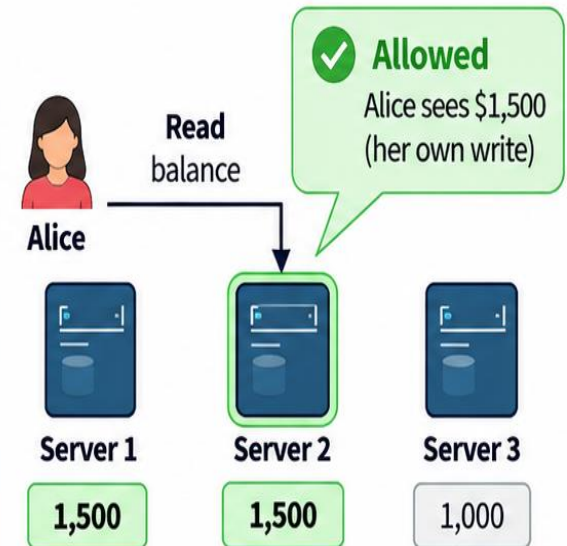
2

Replication updates Server 2
(Server 3 not yet updated)



3

Alice reads from Server 2



time

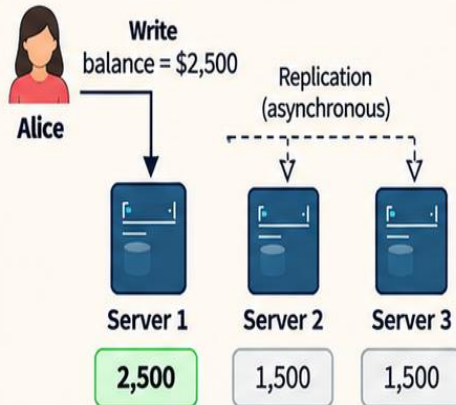
Monotonic Read Consistency

A client is guaranteed to never see older values: once it has read a value, all its subsequent reads will see the same value or a more recent one.

Example: Bob reads a balance of \$2,500. If he reads again, he will still see \$2,500 even if some replicas have not yet been updated.

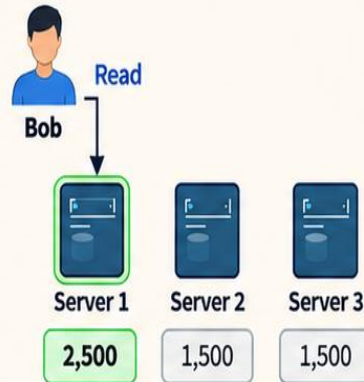
1 Update and replication

Alice updates the balance to \$2,500.



2 First read (sees new value)

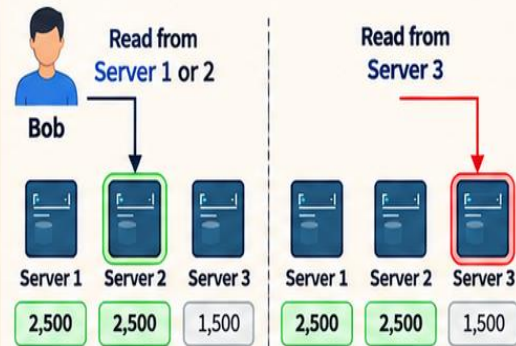
Bob reads from Server 1 and sees \$2,500.



✓ **Allowed**
Bob sees \$2,500.

3 Second read (must not see an older value)

If Bob reads again, he must not see an older value (e.g., \$1,500).



✓ **Allowed**
Bob sees \$2,500 (same value).

✗ **Not allowed**
Bob cannot see \$1,500 after having already seen \$2,500.

4 After replication

Once replication completes, all servers have the new value and reads are allowed from any server.



✓ **Allowed**
Bob sees \$2,500 from any server.

time →

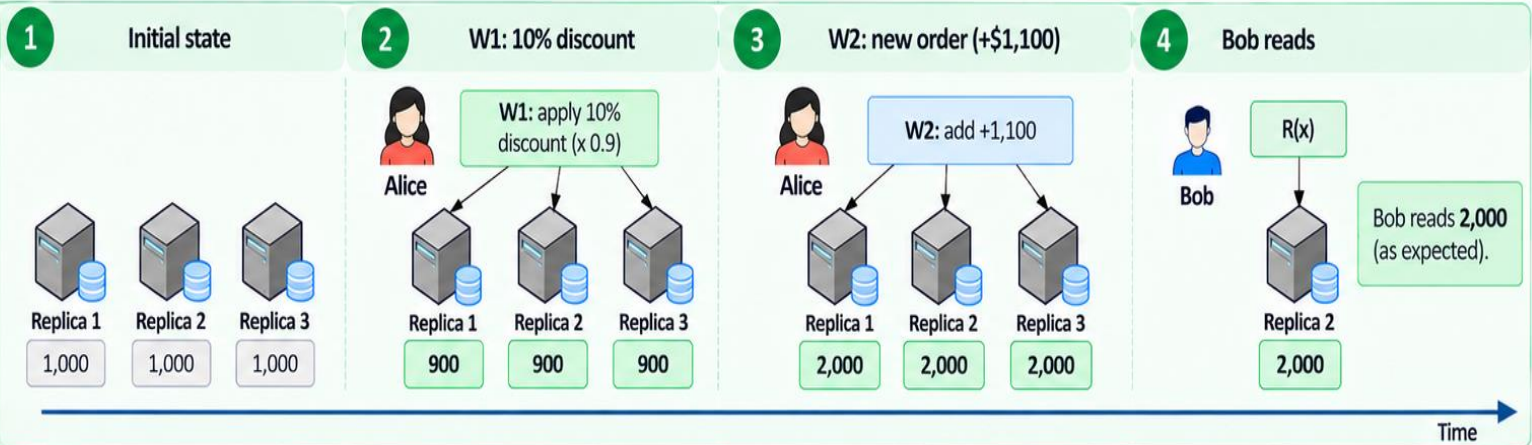
Monotonic Write Consistency: Example with Asynchronous Replication

Alice applies a 10% discount to all customers' balances, then processes a new order for Charlie (+\$1,100).
All clients must see these writes in the same order (1,000 → 900 → 2,000), even if replicas are updated asynchronously.

Correct order (at all replicas)

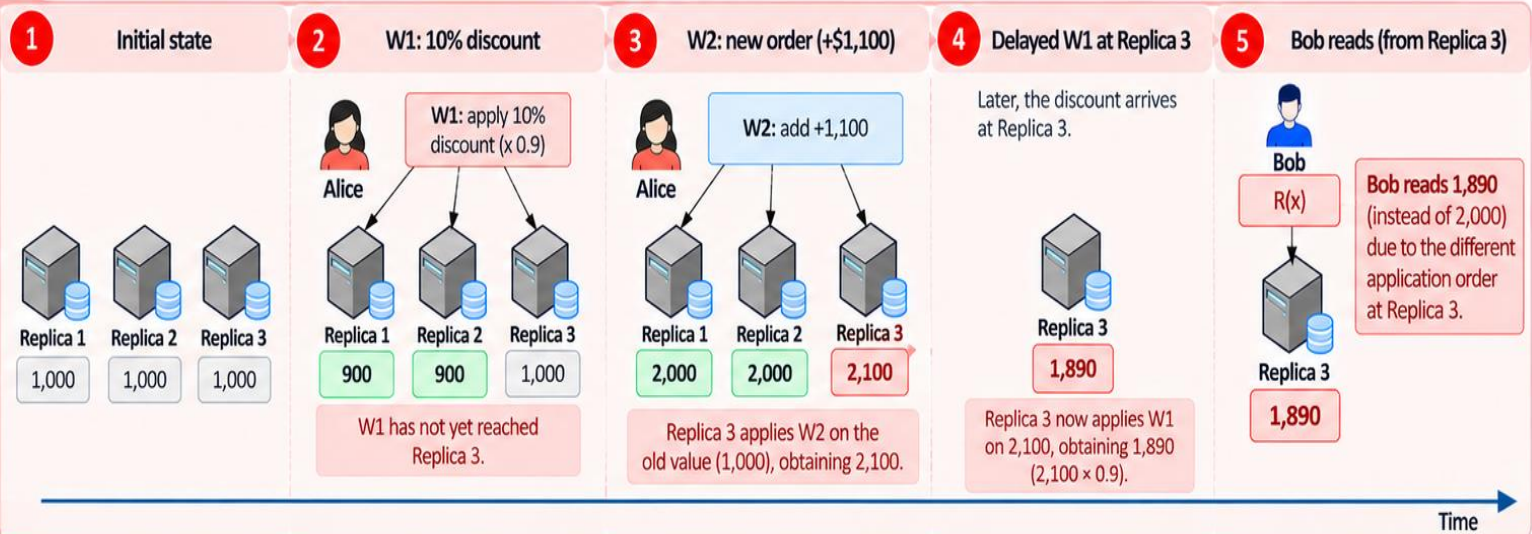
All replicas apply Alice's writes in the same order (discount → new order).

Charlie's balance evolves as:
1,000 → 900 → 2,000.



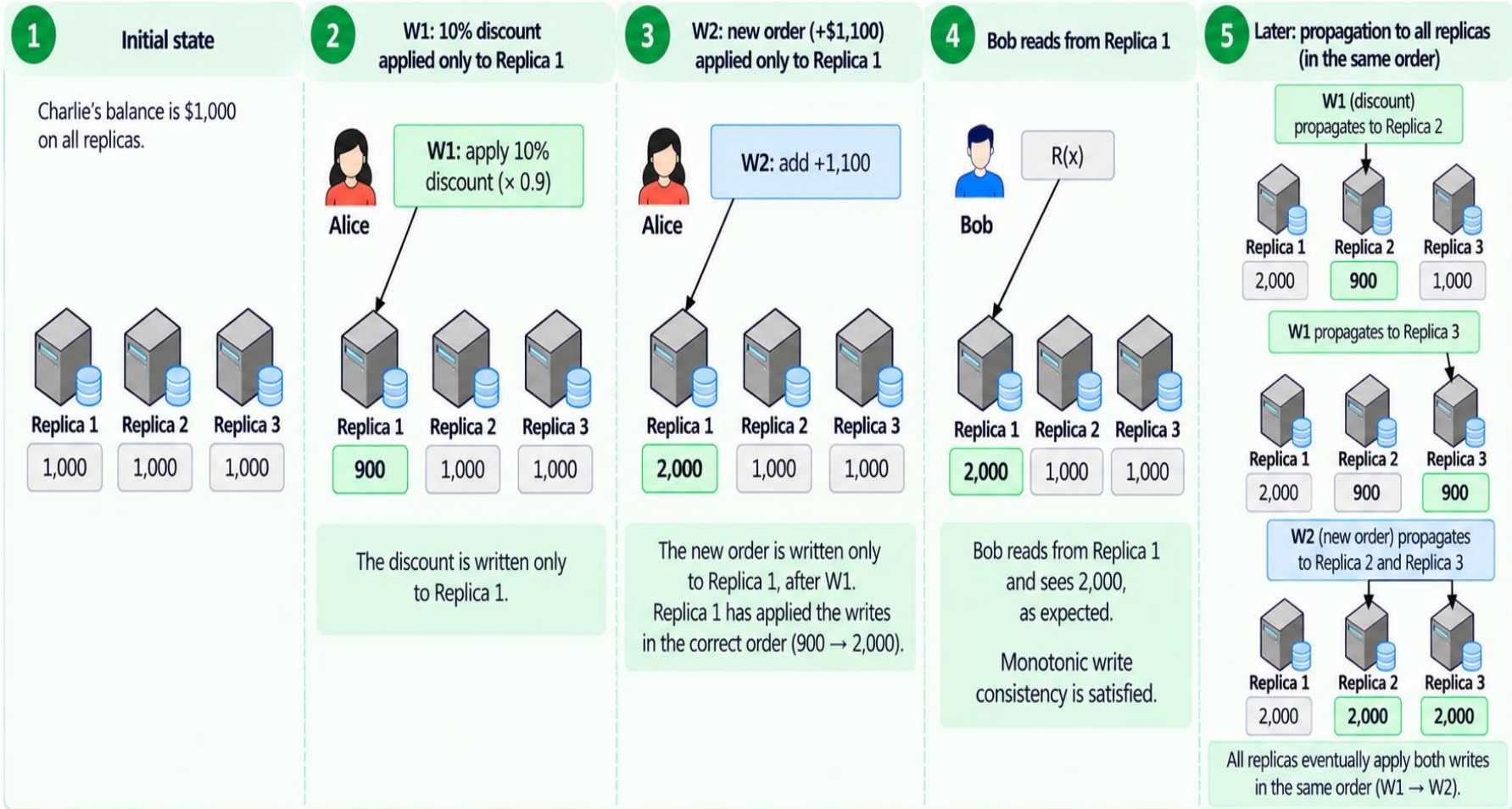
Different order at one replica (inconsistent)

Due to asynchronous replication, Replica 3 receives W2 before W1. Replica 3 applies the new order on the old value (1,000), obtaining 2,100, and later applies the discount, obtaining 1,890. A client reading from Replica 3 observes 1,890 instead of 2,000.



Monotonic Write Consistency: Example with Propagation to a Single Replica

Writes may reach replicas at different times. As long as each replica applies Alice's writes in the same order ($W1 \rightarrow W2$), clients read a consistent value (e.g., from Replica 1) and monotonic write consistency is preserved.



Time

Causal Consistency Example (i)

Let suppose the following wall of a social network with comments and replies.

PIETRO: What a beautiful day!

MIKE: Lucky you! Here it is raining cats and dogs!

Pippo: Also in Pisa it is a good day

LUCA: I passed the examination!

MIKE: Great, you have to pay a beer to all the group tonight!

MIKE: check my last article on www.mike.org

Mike and Pippo's posts are causally related to Pietro's post, but there is no causality between them!



Accepted View Order

MIKE: check my last article on www.mike.org

LUCA: I passed the examination!

MIKE: Great, you have to pay a beer to all the group tonight!

PIETRO: What a beautiful day!

MIKE: Lucky you! Here it is raining cats and dogs!

Pippo: Also in Pisa it is a good day

Not Accepted View Order

MIKE: check my last article on www.mike.org

MIKE: Lucky you! Here it is raining cats and dogs!

MIKE: Great, you have to pay a beer to all the group tonight!

PIETRO: What a beautiful day!

LUCA: I passed the examination!

Pippo: Also in Pisa it is a good day

Causal Consistency Example (ii)

Let suppose the following wall of a social network with comments and replies.

PIETRO: What a beautiful day!

MIKE: Lucky you! Here it is raining cats and dogs!

Pippo: Also in Pisa it is a good day

LUCA: I passed the examination!

MIKE: Great, you have to pay a beer to all the group tonight!

MIKE: check my last article on www.mike.org

Mike and Pippo's posts are causally related to Pietro's post, but there is no causality between them!



Accepted View Order

MIKE: check my last article on www.mike.org

LUCA: I passed the examination!

MIKE: Great, you have to pay a beer to all the group tonight!

PIETRO: What a beautiful day!

Pippo: Also in Pisa it is a good day

MIKE: Lucky you!.....

Not Accepted View Order

MIKE: check my last article on www.mike.org

MIKE: Lucky you! Here it is raining cats and dogs!

MIKE: Great, you have to pay a beer to all the group tonight!

PIETRO: What a beautiful day!

LUCA: I passed the examination!

Pippo: Also in Pisa it is a good day

Causal Consistency Example (iii)

Let suppose the following wall of a social network with comments and replies.

PIETRO: What a beautiful day!

MIKE: Lucky you! Here it is raining cats and dogs!

Pippo: Also in Pisa it is an awful da!

LUCA: I passed the examination!

MIKE: Great, you have to pay a beer to all the group tonight!

MIKE: check my last article on www.mike.org

Pippo's post is causally related to Mikes's post.

Both posts are causally related to Pietro's post.

t
i
m
e



Accepted View Order

MIKE: check my last article on www.mike.org

LUCA: I passed the examination!

MIKE: Great, you have to pay a beer to all the group tonight!

PIETRO: What a beautiful day!

MIKE: Lucky you! Here it is raining cats and dogs!

Pippo: Also in Pisa it is an awful day!

Not Accepted View Order

MIKE: check my last article on www.mike.org

MIKE: Lucky you! Here it is raining cats and dogs!

MIKE: Great, you have to pay a beer to all the group tonight!

PIETRO: What a beautiful day!

LUCA: I passed the examination!

Pippo: Also in Pisa it is an awful day

Causal vs Monotonic Write Consistency

Let suppose the following wall of a social network with comments and replies.

PIETRO: What a beautiful day!

MIKE: Lucky you! Here it is raining cats and dogs!

Pippo: Also in Pisa it is a good day

LUCA: I passed the examination!

MIKE: Great, you have to pay a beer to all the group tonight!

MIKE: check my last article on www.mike.org



Accepted View Order *for causal consistency*

MIKE: check my last article on www.mike.org

LUCA: I passed the examination!

MIKE: Great, you have to pay a beer to all the group tonight!

PIETRO: What a beautiful day!

MIKE: Lucky you! Here it is raining cats and dogs!

Pippo: Also in Pisa it is a good day

Accepted view for Monotonic Write Consistency

PIETRO: What a beautiful day!

MIKE: Lucky you! Here it is raining cats and dogs!

Pippo: Also in Pisa it is a good day

LUCA: I passed the examination!

MIKE: Great, you have to pay a beer to all the group tonight!

MIKE: check my last article on www.mike.org

Suggested Readings

Chapter 2 of the book “Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015”

Chapter 4 of the book ““Andreas Meier, Michael Kaufmann , SQL & NoSQL databases : models, languages, consistency options and architectures for big data management, 2019”

<https://storage.googleapis.com/pub-tools-public-publication-data/pdf/45855.pdf>

Brewer, Eric. "Pushing the cap: Strategies for consistency and availability." Computer 45.2 (2012): 23-29.

Chandra, Deka Ganesh. "BASE analysis of NoSQL database." Future Generation Computer Systems 52 (2015): 13-21.

<http://sergeiturukin.com/2017/06/29/eventual-consistency.html>